

Introduction to Hibernate

Java objects and relational tables

Consider this Java class:

```
public class Student {  
  
    private Long id;  
    private String name;  
    private String email;  
    private Integer age;  
  
}
```

The equivalent relational representation might be:

```
CREATE TABLE students (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE,  
    age INT  
);
```

The object and row contain similar information, but they are fundamentally different structures.

Java understands

- Classes
- Objects
- References
- Inheritance
- Encapsulation

- Collections
- Object identity
- Method behaviour

A relational database understands

- Tables
- Rows
- Columns
- Primary keys
- Foreign keys
- Constraints
- Joins
- SQL data types

Therefore, Java cannot directly save an object into MySQL.

The object must first be translated into SQL-compatible values.

Persistence using Spring JDBC

With Spring JDBC, an insertion may look like this:

```
public int save(Student student) {  
  
    String sql = ""  
        INSERT INTO students(name, email, age)  
        VALUES (?, ?, ?)  
        "";  
  
    return jdbcTemplate.update(  
        sql,  
        student.getName(),  
        student.getEmail(),  
    );  
}
```

```
        student.getAge()  
    );  
}
```

This is much cleaner than raw JDBC, but the developer still performs the translation manually.

Spring JDBC manages much of the JDBC infrastructure, but it does not understand the semantic relationship between the `Student` class and the `students` table.

Manual SQL writing

For every persistence operation, SQL must be written manually.

Insert

```
INSERT INTO students(name, email, age)  
VALUES (?, ?, ?);
```

Select

```
SELECT id, name, email, age  
FROM students  
WHERE id = ?;
```

Update

```
UPDATE students  
SET name = ?, email = ?, age = ?  
WHERE id = ?;
```

Delete

```
DELETE FROM students
WHERE id = ?;
```

This is not necessarily bad. SQL gives precise control.

The problem appears when an application has:

- Hundreds of entities
- Many relationships
- Repeated CRUD operations

The amount of repetitive translation code grows rapidly.

Manual parameter binding

Spring JDBC reduces direct `PreparedStatement` handling, but values must still be passed in the correct order.

```
jdbcTemplate.update(
    sql,
    student.getName(),    // first ?
    student.getEmail(),   // second ?
    student.getAge(),     // third ?
    student.getId()       // fourth ?
);
```

The programmer must ensure that:

```
First ? = name
Second ? = email
Third ? = age
Fourth ? = id
```

A wrong order may not cause a compilation error.

Manual row-to-object mapping

When reading data, the opposite translation is needed.

```
private final RowMapper<Student> studentRowMapper =  
    (resultSet, rowNumber) -> {  
  
        Student student = new Student();  
  
        student.setId(resultSet.getLong("id"));  
        student.setName(resultSet.getString("name"));  
        student.setEmail(resultSet.getString("email"));  
        student.setAge(resultSet.getInt("age"));  
  
        return student;  
    };
```

Object-relational impedance mismatch

The differences between object models and relational models are collectively called **object-relational impedance mismatch**.

Introduction to Object-Relational Mapping

Object-Relational Mapping, or ORM, is the process of defining how:

```
Java classes      map to database tables  
Java objects      map to database rows  
Java fields       map to database columns  
Java references   map to foreign-key relationships  
Java collections  map to related rows
```

For Example :

```
public class Student {
```

```
private Long id;

private String name;
}
```

This metadata tells the ORM framework:

```
Student class → students table
id field      → id column
name field    → student_name column
```

The mapping is declared once and reused during `inserts`, `selects`, `updates` and `deletes`.

Responsibilities of an ORM framework

An ORM framework normally handles several responsibilities.

Mapping metadata

It understands how classes and attributes correspond to tables and columns.

SQL generation

It can generate SQL based on entity operations.

Parameter binding

It extracts values from the entity and binds them to JDBC parameters.

Result-set mapping

It converts database rows back into entity objects.

Relationship management

It converts object references and collections into foreign-key and join-table operations.

Caching

It may avoid unnecessary database access through first-level and optional second-level caches.

Transaction integration

It coordinates entity operations with database transactions.

Introduction to Hibernate

Hibernate is an ORM framework for Java and JVM applications.

Hibernate operates above JDBC.

Application



Hibernate



JDBC



JDBC Driver



Database

JPA as a specification

JPA originally meant **Java Persistence API**.

The current standard is named **Jakarta Persistence**, though developers still commonly use the term JPA.

It defines contracts such as:

```
jakarta.persistence.Entity  
jakarta.persistence.Id  
jakarta.persistence.EntityManager
```

```
jakarta.persistence.EntityManagerFactory  
jakarta.persistence.PersistenceContext
```

But a specification does not perform the work by itself.

An implementation is required.

Hibernate as a JPA provider

Hibernate implements the Jakarta Persistence specification.

Therefore, when application code uses:

```
import jakarta.persistence.EntityManager;
```

the interface is defined by Jakarta Persistence, but the runtime object may be implemented by Hibernate.

This resembles other interface-and-implementation relationships:

```
List interface  
↓  
ArrayList implementation
```

Jakarta Persistence defines the API. Hibernate supplies the implementation.

Other JPA providers have historically included EclipseLink and OpenJPA, but Spring Boot commonly uses Hibernate when the JPA starter is present. Spring Boot's JPA starter includes Hibernate, Spring Data JPA and Spring ORM.

Setting Up Hibernate with Spring Boot

1. Required dependencies

Create a Spring Boot project with:

- **Spring Data JPA** (We are including `spring-boot-starter-data-jpa` for configuration and dependencies, but we are not initially using `JpaRepository`. Spring Data repositories would otherwise hide many of these operations.)
- **MySQL Driver**

Creating and using Student database

Hibernate can manage tables, but for a basic MySQL setup, create the database first:

```
CREATE DATABASE student_db;  
  
USE student_db;
```

DataSource configuration

In `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/hibernate_demo  
spring.datasource.username=root  
spring.datasource.password=your_password
```

Spring Boot uses these properties to configure a `DataSource`.

The `DataSource` supplies JDBC connections to Hibernate.

Schema generation with `ddl-auto`

For learning:

```
spring.jpa.hibernate.ddl-auto=update
```

Common values are:

`none`

Hibernate performs no schema action.

```
spring.jpa.hibernate.ddl-auto=none
```

validate

Hibernate checks whether the existing schema matches the entity mappings.

```
spring.jpa.hibernate.ddl-auto=validate
```

It does not create or modify tables.

update

Hibernate attempts to update the schema to match the mappings.

```
spring.jpa.hibernate.ddl-auto=update
```

Convenient for demonstrations and early development, but it should not be treated as a production migration system.

create

Hibernate drops and recreates the mapped schema when the application starts.

```
spring.jpa.hibernate.ddl-auto=create
```

Existing table data may be lost.

create-drop

Hibernate creates the schema at startup and drops it when the persistence factory closes.

```
spring.jpa.hibernate.ddl-auto=create-drop
```

Spring Boot exposes `spring.jpa.hibernate.ddl-auto`, and current documentation lists `none`, `validate`, `update`, `create` and `create-drop` as its supported modes.

In production systems, schema changes are usually managed using tools such as Flyway or Liquibase rather than depending on `update`.

SQL logging

For learning, enable generated SQL:

```
spring.jpa.show-sql=true
```

Format the SQL:

```
spring.jpa.properties.hibernate.format_sql=true
```

Spring Boot Project Structure

Use this structure:

```
hibernate-demo
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com.coderarmy.hibernate
│   │   │   │   ├── HibernateDemoApplication.java
│   │   │   │   ├── entity
│   │   │   │   │   ├── Student.java
│   │   │   │   │   └── Address.java
│   │   │   │   ├── repository
│   │   │   │   │   └── StudentPersistenceRepository.java
│   │   │   │   ├── service
│   │   │   │   │   └── StudentService.java
│   │   │   │   └── controller
│   │   │   │       └── StudentController.java
│   │   └── resources
│   │       └── application.properties
```

Initial entity

```
package com.coderarmy.hibernate.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private Integer age;

    public Student() {
    }

    public Student(
        String name,
        String email,
        Integer age
    ) {
        this.name = name;
        this.email = email;
        this.age = age;
    }

    // getter & setter
}
```

Repository

```
@Repository
public class StudentPersistenceRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public void persist(Student student) {
        entityManager.persist(student);
    }

    public Student findById(Long id) {
        return entityManager.find(Student.class, id);
    }

    public void remove(Student student) {
        entityManager.remove(student);
    }
}
```

The standard JPA interface through which application code asks the persistence provider to manage and retrieve entities.

5.2 Create with `persist()`

```
@Transactional
public Long createStudent(
    String name,
    String email,
    Integer age
) {
    Student student =
        new Student(name, email, age);
}
```

```
repository.persist(student);

return student.getId();
}
```

5.3 Read with `find()`

```
@Transactional(readOnly = true)
public Student getStudent(Long id) {
    return repository.findById(id);
}
```

5.4 Update through a managed entity

```
@Transactional
public void updateStudent(Long id, String newName) {

    Student student = repository.findById(id);

    if (student == null) {
        throw new IllegalArgumentException(
            "Student not found: " + id
        );
    }

    student.setName(newName);
}
```

Pause on what is missing:

```
entityManager.update(student);
repository.save(student);
```

Neither is required for a managed entity.

Do not fully explain dirty checking here. Create curiosity:

Hibernate somehow knows that the loaded object changed and generates an update before the transaction commits. The persistence-context section will show exactly how.

5.5 Delete with `remove()`

```
@Transactional
public void deleteStudent(Long id) {

    Student student = repository.findById(id);

    if (student == null) {
        return;
    }

    repository.remove(student);
}
```

Explain:

- `remove()` operates on a managed entity.
- Hibernate marks that entity for deletion.
- The delete is synchronized with the database during flushing.

Initial entity

```
package com.coderarmy.hibernate.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
```

```
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(
        name = "student_name",
        nullable = false,
        length = 100
    )
    private String name;

    @Column(
        nullable = false,
        unique = true,
        length = 150
    )
    private String email;

    private Integer age;

    protected Student() {
    }

    public Student(
        String name,
        String email,
        Integer age
    ) {
        this.name = name;
        this.email = email;
        this.age = age;
    }
}
```



```
// getter & setter  
}
```

Important options include:

```
@Column(  
    name = "student_name",  
    nullable = false,  
    unique = false,  
    length = 100,  
    insertable = true,  
    updatable = true  
)  
private String name;  
  
//name  
Controls the mapped column name.  
  
//nullable  
Used in schema generation to indicate whether the column may  
be null.  
  
//unique  
Declares a single-column uniqueness constraint during schema  
generation.  
  
//length  
Provides the expected string-column length for generated sche  
ma.  
  
//insertable  
Controls whether the column appears in generated insert state  
ments.  
  
//updatable
```

Controls whether the column appears in generated update statements.

Use `BigDecimal` when decimal precision matters.

For example:

```
@Column(precision = 5, scale = 2)
private BigDecimal percentage;
```

Conceptually:

```
precision = total number of digits
scale      = digits after decimal point
```

For:

98.75

The precision is 4 and the scale is 2.

Date and time mapping

Modern Java date/time classes should normally be preferred.

```
private LocalDate dateOfBirth;
```

Maps conceptually to:

DATE

2000-08-21

```
private LocalTime classTime;
```

Maps conceptually to:

TIME

```
private LocalDateTime createdAt;
```

Maps conceptually to:

TIMESTAMP

2026-07-25T10:30:00

Enum mapping

Consider:

```
public enum StudentStatus {  
    ACTIVE,  
    INACTIVE,  
    SUSPENDED  
}
```

Entity field:

```
@Enumerated(EnumType.STRING)  
private StudentStatus status;
```

Database values:

ACTIVE
INACTIVE
SUSPENDED

Large objects

Large text

```
@Lob
private String profileDescription;
```

Binary data

```
@Lob
private byte[] profileImage;
```

`@Lob` instructs the provider to use a database-native large-object mapping.

For real production systems, storing large files directly inside the database should be an intentional architectural decision. Often, the database stores file metadata and an object-storage URL instead.

@Transient

Sometimes an entity needs a field that should not be persisted.

```
@Transient
private String displayName;
```

Hibernate ignores it.

Attribute converters

Suppose the database stores:

```
Y
N
```

But Java should use:

```
Boolean
```

Create a converter:

```
package com.coderarmy.hibernate.converter;

import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;

@Converter
public class BooleanToYesNoConverter
    implements AttributeConverter<Boolean, String> {

    @Override
    public String convertToDatabaseColumn(
        Boolean attribute
    ) {
        if (attribute == null) {
            return null;
        }

        return attribute ? "Y" : "N";
    }

    @Override
    public Boolean convertToEntityAttribute(
        String databaseValue
    ) {
        if (databaseValue == null) {
            return null;
        }

        return "Y".equals(databaseValue);
    }
}
```

Use it:

```
@Convert(converter = BooleanToYesNoConverter.class)
private Boolean active;
```

Embeddable Objects and Value Types

1. Entity types versus value types

Consider a student and an address.

Student

A student has independent identity.

The student continues to exist even if the address changes.

Address

An address often has no independent identity in the domain.

House number
Street
City
State
Postal code

It exists as part of another object.

Student owns Address

That makes `Address` a good candidate for a value type.

Hibernate classifies value types as values whose lifecycle is owned by an entity, whereas entities have independent identity.

2. `@Embeddable`

Define the value type:

```
package com.coderarmy.hibernate.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;

@Embeddable
public class Address {

    @Column(name = "address_line")
    private String addressLine;

    private String city;

    private String state;

    @Column(name = "postal_code")
    private String postalCode;

    protected Address() {
    }

    public Address(
        String addressLine,
        String city,
        String state,
        String postalCode
    ) {
        this.addressLine = addressLine;
        this.city = city;
        this.state = state;
        this.postalCode = postalCode;
    }
}
```

`@Embeddable` means:

This class does not represent an independent entity. Its state may be embedded inside an owning entity.

An embeddable does not have its own `@Id`. Its state is stored as an intrinsic part of its owner.

`@Embedded`

Use the value inside `Student`:

```
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Embedded
    private Address address;
}
```

The table may become:

```
students
├─ id
├─ name
├─ address_line
├─ city
├─ state
└─ postal_code
```

There is no separate address table by default.

The object structure:

Student
└─ Address

Is flattened into:

students row
└─ student fields
└─ address fields

`@Embedded` declares an entity attribute whose value is an embeddable object.

Why use an embeddable instead of separate fields?

Without an embeddable:

```
private String addressLine;  
private String city;  
private String state;  
private String postalCode;
```

This works, but the entity begins to contain unrelated groups of primitive fields.

With an embeddable The Java model becomes more expressive.

Multiple embedded objects

Suppose a student has:

```
@Embedded  
private Address currentAddress;  
  
@Embedded  
private Address permanentAddress;
```

Both embeddables contain fields named:

```
address_line
city
state
postal_code
```

This would create conflicting column mappings.

We must override them.

@AttributeOverride

```
@Embedded
@AttributeOverrides({
    @AttributeOverride(
        name = "addressLine",
        column = @Column(
            name = "current_address_line"
        )
    ),
    @AttributeOverride(
        name = "city",
        column = @Column(
            name = "current_city"
        )
    ),
    @AttributeOverride(
        name = "state",
        column = @Column(
            name = "current_state"
        )
    ),
    @AttributeOverride(
        name = "postalCode",
        column = @Column(
            name = "current_postal_code"
        )
    )
})
```

```

    )
}
private Address currentAddress;

```

same for permanent address.

The resulting table contains:

```

students
├─ id
├─ current_address_line
├─ current_city
├─ current_state
├─ current_postal_code
├─ permanent_address_line
├─ permanent_city
├─ permanent_state
└─ permanent_postal_code

```

`@AttributeOverride` replaces a basic mapping originally declared by the embeddable for a particular embedding location

@ElementCollection

An entity may own a collection of value types.

For example:

```

private Set<String> skills;

```

A collection cannot normally fit into one relational column, so it is stored in a separate table.

```

@ElementCollection
@CollectionTable(
    name = "student_skills",

```

```

        joinColumns = @JoinColumn(name = "student_id")
    )
    @Column(name = "skill")
    private Set<String> skills =
        new HashSet<>();

```

Possible database structure:

```

students
├── id
└── name

```

```

student_skills
├── student_id
└── skill

```

The values in `student_skills` are not independent entities.

There is no:

```

Skill skill = entityManager.find(Skill.class, id);

```

Each string belongs to the owning `Student`.

Element collection of embeddables

You may also store a collection of value objects.

```

@Embeddable
public class PhoneNumber {

    private String countryCode;
    private String number;

    protected PhoneNumber() {
    }
}

```

```
public PhoneNumber(
    String countryCode,
    String number
) {
    this.countryCode = countryCode;
    this.number = number;
}
}
```

Inside `Student`:

```
@ElementCollection
@CollectionTable(
    name = "student_phone_numbers",
    joinColumns = @JoinColumn(name = "student_id")
)
private Set<PhoneNumber> phoneNumbers =
    new HashSet<>();
```

Table:

```
student_phone_numbers
├─ student_id
├─ country_code
└─ number
```

`@ElementCollection` maps collections of basic or embeddable values, while `@CollectionTable` controls the separate table used to store them.

Entity association versus embedded value

This is an important modelling decision.

Use an entity when the object has independent identity

Example:

Student
Department
Course
Teacher

Coder Army

A department exists independently of one particular student.

It may be:

- Shared by many students
 - Queried independently
 - Updated independently
 - Referenced by an identifier
 - Managed through its own lifecycle
-